

Lecture 11: Rethinking Generalization: Modern Phenomena in Overparameterized Models

Readings: ISL (Ch. 10), ESL (Ch. 11), Bach (Ch. 12); code

Soon Hoe Lim

September 30, 2025

Outline

- ➊ Motivation: Successes of Overparametrized Models in Deep Learning
- ➋ Implicit Regularization
- ➌ Double Descent
- ➍ Benign Overfitting
- ➎ Scaling Laws
- ➏ Key Takeaways
- ➐ Exercises
- ➑ Appendix

The Classical View vs Modern Reality

Classical Machine Learning Wisdom:

- ▶ A model is **overparameterized** if the number of learnable parameters $p > N$, in which case there are infinitely many interpolating solutions.
- ▶ Classical wisdom: Generalization is governed by the **bias–variance tradeoff**, and overparametrized models should overfit severely.
- ▶ More parameters $\Rightarrow$ worse generalization.

Modern Deep Learning Reality:

- ▶ State-of-the-art deep learning models have far more parameters than training examples.
- ▶ They achieve near-zero training error (*perfect interpolation*).
- ▶ Yet they generalize extremely well on unseen data.

Understanding Deep Learning (Still) Requires Rethinking Generalization

Zhang et. al. (2017): Deep neural nets can memorize random labels (noise) *and* still generalize on real data.

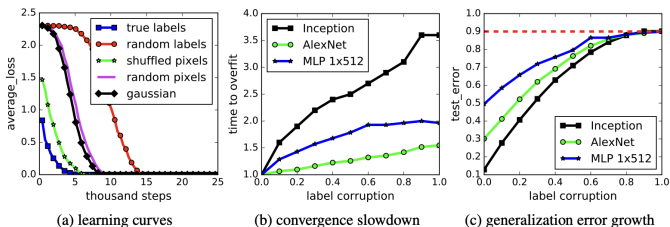


Figure 1: Fitting random labels and random pixels on CIFAR10. (a) shows the training loss of various experiment settings decaying with the training steps. (b) shows the relative convergence time with different label corruption ratio. (c) shows the test error (also the generalization error since training error is 0) under different label corruptions.

❓ Why can overparameterized models generalize so well, despite classical theory predicting overfitting?

Why Overparameterization Can Help?

- ▶ **Implicit Regularization:** Optimization (e.g., gradient descent) naturally biases toward "simple" solutions.
- ▶ **Double Descent:** Test error can show multiple descents (e.g., as a function of model size, data size, or training time). How do we reconcile classical ML theory with the apparent success of overparameterized models?
- ▶ **Benign Overfitting:** Provides a theoretical explanation for double descent — even interpolating models can generalize due to implicit biases.
- ▶ **Scaling Laws:** Performance improves predictably with more parameters, data, and compute.
- ▶ **Infinite-Width Limit:** Gradient descent can exhibit global convergence, despite nonconvexity (via the Neural Tangent Kernel perspective).

💡 Today we discuss the **successes** of overparameterized models, focusing on the lens of *implicit regularization* and *double descent*. In the next lecture, we will examine their **limitations and reliability issues**—both for overparameterized and other modern models.

Implicit Regularization by Optimization

- ▶ Classical generalization relies on **explicit regularization**:

$$\hat{R}_N(\theta) + \lambda \|\theta\|^2 \quad (\text{ridge}), \quad \hat{R}_N(\theta) + \lambda \|\theta\|_1 \quad (\text{lasso}).$$

- ▶ **Empirical observation:** In overparameterized settings, even without explicit penalties, GD biases toward “simple” solutions.
- ▶ This **implicit bias** can be viewed as “algorithmic regularization”, which depends on:
 - ▶ initialization (e.g., $\theta^{(0)} = 0$),
 - ▶ learning rate η ,
 - ▶ algorithm variant (GD, SGD, Adam, etc.),
 - ▶ any manipulation like dropout or batch normalization.
- ▶ The combination of initialization and the *architecture* of the predictor family further constrain model capacity implicitly.

💡 Classical guarantees rely on *explicit* penalties (ridge, lasso), whereas GD provides an *implicit* bias even without such terms. How can we understand the implicit bias of GD?

Linear Models Trained with Gradient Descent

Given the training data $D_N = \{(x_i, y_i)\}_{i=1, \dots, N}$, $x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$, the linear model is $f_\theta(x) = \theta^\top x = \hat{y}$. Here, $p = d$ is the number of learnable parameters.

- ▶ **Loss function (squared loss):** for prediction $\hat{y}$ and true label y ,

$$L(\hat{y}, y) = (\hat{y} - y)^2.$$

- ▶ **Expected (population) risk** (with $\mathbb{P}$ the ground truth distribution):

$$R(\theta) = \mathbb{E}_{(x,y) \sim \mathbb{P}} [L(x^\top \theta, y)].$$

- ▶ **Empirical (training) risk:**

$$\hat{R}(\theta) = \hat{R}_N(\theta) = \frac{1}{N} \sum_{i=1}^N L(x_i^\top \theta, y_i).$$

- ▶ **Excess risk:**

$$R(\theta) - R^*, \quad \text{where } R^* = \min_{\theta} R(\theta).$$

Setup and Assumptions

Let's focus on the ordinary least square (OLS) regression¹:

- ▶ **Design matrix:** $\mathbf{X} \in \mathbb{R}^{N \times d}$, with rows $x_i^\top$ (the training inputs).
- ▶ **Response vector:** $\mathbf{y} \in \mathbb{R}^N$, with entries y_i .
- ▶ **Empirical risk (least squares):**

$$\hat{R}(\theta) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\theta\|_2^2.$$

- ▶ **Overparameterized regime:** $d > N$, so the system is underdetermined. There may be infinitely many solutions (interpolators) θ with $\mathbf{X}\theta = \mathbf{y}$.
- ▶ **Rank assumption:** $\text{rank}(\mathbf{X}) = N$ (so the Gram matrix $K = \mathbf{X}\mathbf{X}^\top$ is invertible).
- ▶ **Initialization & step size:** $\theta^{(0)} = 0$, and $0 < \eta < 1/\lambda_{\max}$, where $\lambda_{\max}$ is the largest eigenvalue of $\frac{1}{N}\mathbf{X}\mathbf{X}^\top$.

¹We can also study the implicit bias of GD in classification and nonconvex settings but the analysis is more complicated (see Ch. 12.1 in Bach).

Gradient Descent Update and Main Theorem

GD update: $\theta^{(t)} = \theta^{(t-1)} - \eta \nabla \hat{R}_N(\theta^{(t-1)}) = \theta^{(t-1)} - \frac{\eta}{N} \mathbf{X}^\top (\mathbf{X}\theta^{(t-1)} - \mathbf{y})$.

In Exercise 10.4, we have shown the following result:

Theorem 1: Implicit ℓ_2 -Regularization of GD

With $\theta^{(0)} = 0$ and $0 < \eta < 1/\lambda_{\max}$,

$$\theta^{(\infty)} = \lim_{t \rightarrow \infty} \theta^{(t)} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top)^{-1} \mathbf{y}.$$

Moreover, $\theta^{(\infty)}$ is the *minimum ℓ_2 -norm least-squares solution*, i.e.:

$$\theta^{(\infty)} = \arg \min_{\theta \in \mathbb{R}^d} \{ \|\theta\|_2 : \theta \in \arg \min_{\vartheta} \|\mathbf{X}\vartheta - \mathbf{y}\|_2 \}.$$

- ▶ Therefore, $\mathbf{X}\theta^{(t)} \rightarrow \mathbf{y}$ as $t \rightarrow \infty$. This is a linear convergence result, and the convergence rate depends on the condition number of the Gram matrix.
- ▶ No Gaussian assumption is needed for the convergence/limit identity.
- ▶ $\mathbf{X}^+ := \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top)^{-1}$ is also called the (Moore-Penrose) pseudoinverse. This is a right inverse, as $\mathbf{X}\mathbf{X}^+ = I_N$.

GD Trajectories Stay in $\text{span}(\mathbf{X}^\top)$

- ▶ With $\theta^{(0)} = 0$, each GD update

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{1}{N} \mathbf{X}^\top (\mathbf{X} \theta^{(t)} - \mathbf{y})$$

adds a vector in $\text{span}(\mathbf{X}^\top)$.

- ▶ By induction:

$$\theta^{(t)} \in \text{span}(\mathbf{X}^\top), \quad \forall t \geq 0.$$

- ▶ Thus we can write

$$\theta^{(t)} = \mathbf{X}^\top \alpha^{(t)}, \quad \alpha^{(t)} \in \mathbb{R}^N.$$

- ▶ Then

$$\mathbf{X} \theta^{(t)} = \mathbf{X} \mathbf{X}^\top \alpha^{(t)} = K \alpha^{(t)}, \quad K = \mathbf{X} \mathbf{X}^\top.$$

- ▶ Since $\mathbf{X} \theta^{(t)} \rightarrow \mathbf{y}$ and K is invertible, $\alpha^{(t)} \rightarrow K^{-1} \mathbf{y}$, and therefore

$$\theta^{(\infty)} = \mathbf{X}^\top K^{-1} \mathbf{y}.$$

This is an **algorithmic version of the Representer Theorem**: although the parameter space is $\mathbb{R}^d$, GD solutions live in $\text{span}(\mathbf{X}^\top)$, i.e. a space of dimension at most N .

Dual Characterization: Minimum-Norm Interpolator

Primal problem (constrained optimization):

$$\min_{\theta \in \mathbb{R}^d} \frac{1}{2} \|\theta\|_2^2 \quad \text{s.t. } \mathbf{X}\theta = \mathbf{y}.$$

Lagrangian:

$$\mathcal{L}(\theta, \alpha) = \frac{1}{2} \|\theta\|_2^2 + \alpha^\top (\mathbf{y} - \mathbf{X}\theta), \quad \alpha \in \mathbb{R}^N.$$

Dual problem:

$$\sup_{\alpha \in \mathbb{R}^N} g(\alpha), \quad \text{where } g(\alpha) = \inf_{\theta} \mathcal{L}(\theta, \alpha) = \alpha^\top \mathbf{y} - \frac{1}{2} \alpha^\top (\mathbf{X}\mathbf{X}^\top) \alpha,$$

KKT conditions:

$$\nabla_{\theta} \mathcal{L} = 0 \Rightarrow \theta = \mathbf{X}^\top \alpha, \quad \nabla_{\alpha} \mathcal{L} = 0 \Rightarrow \mathbf{X}\mathbf{X}^\top \alpha = \mathbf{y}.$$

Solution: $\hat{\theta} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top)^{-1} \mathbf{y}.$

This minimum-norm solution coincides with the GD limit $\theta^{(\infty)}$!

The Kernel View: Effective Complexity

The dual variables $\alpha \in \mathbb{R}^N$ live in **data space**, not parameter space. This is the kernel viewpoint: solutions are linear combinations of training points.

Recall from the dual/KKT formulation:

$$\hat{\theta} = \mathbf{X}^\top \alpha^*, \quad \alpha^* = (\mathbf{X}\mathbf{X}^\top)^{-1} \mathbf{y}.$$

Prediction for new input x :

$$f(x) = x^\top \hat{\theta} = x^\top \mathbf{X}^\top \alpha^* = \sum_{i=1}^N \alpha_i^* \langle x, x_i \rangle.$$

💡 Although $\hat{\theta} \in \mathbb{R}^d$ with $d \gg N$, the predictor depends only on the $N \times N$ Gram matrix $K = \mathbf{X}\mathbf{X}^\top$. Thus the effective model complexity is $O(N)$, not $O(d)$.

This is exactly the **representer theorem in action**: solutions live in $\text{span}(\mathbf{X}^\top)$, i.e. as linear combinations of training examples. Overparameterization doesn't increase effective complexity when optimization implicitly regularizes.

From Linear Models to Neural Networks

- ▶ In linear least-squares regression, GD from $\theta^{(0)} = 0$ converges to the *minimum ℓ_2 -norm interpolator*. Thus, GD introduces **implicit regularization** towards low ℓ_2 -norm solutions without explicit penalty.
- ▶ This connects directly to the representer theorem and to the kernel view of effective complexity.

How about neural networks?

- ▶ The parameter space is nonlinear and highly nonconvex.
- ▶ Implicit regularization of GD is less well understood in practice.
- ▶ But: in the *infinite-width limit*, the GD dynamics of neural networks reduce to kernel regression with the **Neural Tangent Kernel (NTK)**².

Thus the linear model analysis provides intuition: wide neural networks behave like kernel machines, with the NTK playing the role of the Gram matrix.

²See <https://arxiv.org/abs/1806.07572>.

Infinite-Width Limit and Neural Tangent Kernel

- ▶ In linear regression, GD solutions live in $\text{span}(\mathbf{X}^\top)$, and predictions depend only on the Gram matrix $K = \mathbf{X}\mathbf{X}^\top$.
- ▶ In wide neural networks, the same idea holds with a different kernel: the **Neural Tangent Kernel (NTK)**:

$$K_{\text{NTK}}(x, x') = \nabla_{\theta} f_{\theta}(x)^\top \nabla_{\theta} f_{\theta}(x') \Big|_{\theta=\theta_0},$$

where θ_0 is the random initialization.

💡 Wide neural nets thus behave like kernel machines where the kernel is determined by architecture and initialization. We will not analyze this in detail; see Ch. 12.4 in Bach.

❓ How about **early stopping**? Can we understand its implicit regularization effect for simple models? See Exercise 11.1.

💡 We've seen implicit regularization from **optimization dynamics** so far. There are also implicit/explicit regularization effects from **data perturbations**, such as Gaussian noise injection and Mixup; See Exercise 11.2.

Implicit Regularization: What We Know and Don't Know

Well established for linear models:

- ▶ GD from zero initialization converges to minimum ℓ_2 -norm solution
- ▶ Early stopping provides spectral regularization (your exercise)
- ▶ Kernel view explains effective complexity

Neural networks - empirical observations:

- ▶ Networks seem to find "simpler" solutions among many possibilities
- ▶ Different optimizers exhibit different implicit biases
- ▶ Architecture matters enormously for what constitutes "simple"

Theoretical understanding (limited):

- ▶ Real neural networks are finite-width and perform *feature learning*. The NTK is therefore only an approximation — useful for intuition, but not a full explanation of practical deep learning
- ▶ Multiple competing theories, no consensus

⚠ Our theoretical understanding of implicit regularization in practical neural networks remains incomplete.

Two Complementary Perspectives

Implicit regularization is a general phenomenon: optimization biases solutions toward simpler predictors, even without explicit penalties. Overfitting is avoided by “algorithmic regularization”.

In the overparameterized regime, new behaviors emerge:

- ▶ **Double Descent:** Test error does not just follow the classical U-curve. Around the interpolation threshold, it can explode and then *decrease again*.
- ▶ **Benign Overfitting:** Explains how interpolating models can still generalize — noise is often fit in “harmless” directions.
- ▶ **Infinite-Width Limit:** Neural networks trained by GD behave like kernel machines (NTK), where global convergence can be proven despite nonconvexity. Overparameterization can be a blessing for optimization!
- ▶ **Scaling Laws:** Performance often improves predictably with larger models, more data, and more compute.

We will study double descent in detail for some simple setting, and can only briefly discuss the other behaviors (which are super interesting!).

The Bias-Variance Decomposition

We know from Lecture 1 that the expected prediction error for a predictor $\hat{f}$ (with the true function f^*) at a test point x can be decomposed into:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}[\hat{f}(x)] - f^*(x))^2}_{\text{Bias}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{Variance}} + \sigma^2$$

Classical intuition: Fundamental tradeoff between bias and variance $\Rightarrow$ U-shaped test error.

Classical wisdom: Complex models have low bias but high variance. Simple models have high bias but low variance. Optimal model balances both.

Traditional view:

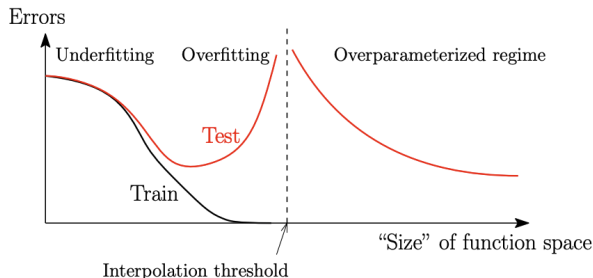
- ▶ Left: Model too simple $\rightarrow$ underfitting $\rightarrow$ high bias
- ▶ Right: Model too complex $\rightarrow$ overfitting $\rightarrow$ high variance
- ▶ Sweet spot: Balanced complexity minimizes total error

Double Descent: Beyond the Classical U-Curve

Number of parameters p ("size" / "capacity" of function space) vs. sample size N

Three regimes:

1. **Underparameterized** ($p < N$): Classical bias-variance tradeoff
2. **Interpolation threshold** ($p \approx N$): Test error explodes
3. **Overparameterized** ($p > N$): Test error decreases again for $p \gg N$!



See a short demo of empirical evidence in Ch. 12.2.2 in Bach.

Empirical Evidence (Belkin et. al., 2019)

The double-descent risk curve manifest with not just neural networks, but also other models such as random forests.

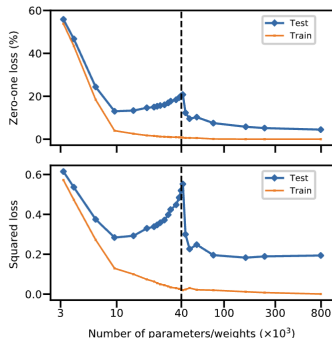


Fig. 3. Double-descent risk curve for a fully connected neural network on MNIST. Shown are training and test risks of a network with a single layer of H hidden units, learned on a subset of MNIST ($n = 4 \cdot 10^3$, $d = 784$, $K = 10$ classes). The number of parameters is $(d+1) \cdot H + (H+1) \cdot K$. The interpolation threshold (black dashed line) is observed at $n \cdot K$.

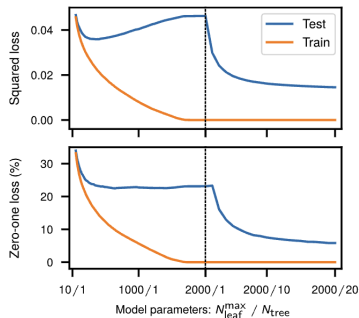


Fig. 4. Double-descent risk curve for random forests on MNIST. The double-descent risk curve is observed for random forests with increasing model complexity trained on a subset of MNIST ($n = 10^4$, 10 classes). Its complexity is controlled by the number of trees N_{tree} and the maximum number of leaves allowed for each tree $N_{\text{leaf}}^{\text{max}}$.

Double Descent in Linear Models

- ▶ To analyze double descent rigorously, we consider simplified models where exact calculations are possible.
 - ▶ Two setups that we can study:
 1. **Gaussian design regression:** inputs $x_i \sim \mathcal{N}(0, I_d)$ with varying dimension d .
 2. **Models with fewer symmetries:** capture a more realistic, full U-shaped curve (not covered here, see Ch. 12.2 in Bach).
 - ▶ In the Gaussian setup:
 - ▶ We can show an **explosion of expected risk** at the interpolation threshold $d \approx N$.
 - ▶ But we do not see a classical U-curve for $d < N$, since the prediction problem itself changes as d grows.
- 💡 We now compute the exact expected risk of the **minimum-norm interpolator** (which is precisely the solution found by GD).

Gaussian Linear Model: Setup and Assumptions

Random (Gaussian) design setting: both $\mathbf{x}$ and $\mathbf{y}$ are considered random, each pair (x_i, y_i) assumed to be drawn i.i.d. from a probability distribution $\mathbb{P}$ on $\mathbb{R}^d \times \mathbb{R}$.

- ▶ Inputs: $x_i \sim \mathcal{N}(0, I_d)$ independently, $i = 1, \dots, N$.
- ▶ Responses: $y_i = x_i^\top \theta^* + \varepsilon_i$, with $\varepsilon_i \sim \text{i.i.d. } \mathcal{N}(0, \sigma^2)$ independent of the x_i .
- ▶ Design matrix: $\mathbf{X} \in \mathbb{R}^{N \times d}$ with rows $x_i^\top$.
- ▶ Kernel (Gram) matrix: $K = \mathbf{X}\mathbf{X}^\top \in \mathbb{R}^{N \times N}$.
- ▶ Non-centered covariance matrix: $\hat{\Sigma} = \frac{1}{N} \mathbf{X}^\top \mathbf{X} \in \mathbb{R}^{d \times d}$.
- ▶ Gradient descent from $\theta^{(0)} = 0$ converges to the minimum-norm interpolator:

$$\hat{\theta} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top)^{-1} \mathbf{y}.$$

- ▶ Since $\Sigma = \mathbb{E}[\mathbf{x}\mathbf{x}^\top] = I_d$, the excess risk simplifies to

$$R(\hat{\theta}) - R^* = \|\hat{\theta} - \theta^*\|_2^2, \quad R^* = \sigma^2.$$

Underparameterized Regime ($d < N$)

- ▶ When $d < N$, the estimator is the OLS estimator θ_{OLS} (see Lecture 2), which is unbiased:

$$\mathbb{E}[\hat{\theta}] = \theta^*.$$

- ▶ Expected excess risk can be shown to be (see Exercise 11.4 (a)):

$$\mathbb{E}[R(\hat{\theta})] - R^* = \sigma^2 \mathbb{E}[\text{tr}((\mathbf{X}^\top \mathbf{X})^{-1})].$$

- ▶ Since $\mathbf{X}^\top \mathbf{X}$ is Wishart-distributed (see blackboard and Appendix), we have, for $N > d + 1$:

$$\mathbb{E}[R(\hat{\theta})] - R^* = \frac{\sigma^2 d}{N - d - 1} = \frac{\sigma^2 d}{N} \cdot \frac{1}{1 - \frac{d+1}{N}}.$$

- ▶ The risk $\rightarrow 0$ as $N \rightarrow \infty$ with fixed d .

Overparameterized Regime ($d \geq N$)

In this case, the Gram matrix $\mathbf{X}\mathbf{X}^\top$ is (almost surely) invertible. The GD minimum-norm interpolator is

$$\hat{\theta} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top)^{-1} \mathbf{y} = \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top)^{-1} \mathbf{X} \theta^* + \mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top)^{-1} \varepsilon.$$

Expected excess risk decomposition (see blackboard):

$$\mathbb{E}[R(\hat{\theta})] - R^* = \mathbb{E}[\|\hat{\theta} - \theta^*\|_2^2] = \text{Bias}^2 + \text{Variance}.$$

► **Variance term:**

$$\sigma^2 \mathbb{E}[\text{tr}((\mathbf{X}\mathbf{X}^\top)^{-1})] = \frac{\sigma^2 N}{d - N - 1}, \quad d > N + 1.$$

► **Bias² term:**

$$\mathbb{E}[\|\mathbf{X}^\top (\mathbf{X}\mathbf{X}^\top)^{-1} \mathbf{X} \theta^* - \theta^*\|_2^2] = \frac{d - N}{d} \|\theta^*\|_2^2.$$

The Main Result: Expected Risk Across Regimes

Collecting all the pieces from before, we obtain the following result.

Theorem 2: Double Descent Risk Curve

For the Gaussian design setting with $x_i \sim \mathcal{N}(0, I_d)$ and $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, the **expected excess risk** of the minimum-norm interpolator is

$$\mathbb{E}[R(\hat{\theta})] - R^* = \begin{cases} \frac{\sigma^2 d}{N - d - 1}, & \text{if } d \leq N - 2, \\ \frac{\sigma^2 N}{d - N - 1} + \frac{d - N}{d} \|\theta^*\|_2^2, & \text{if } d \geq N + 2. \end{cases}$$

Proof.

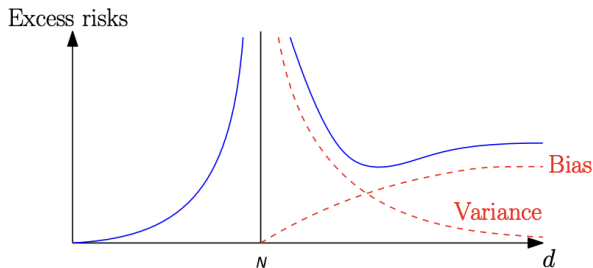
See blackboard. □

Note: At $d = N \pm 1$, the denominators vanish and the expected risk diverges.

Remarks

From Theorem 1, we can observe the following.

- ▶ Blow-up at the interpolation threshold ($d \approx N$).
- ▶ For $d < N$: excess risk is purely from variance.
- ▶ For $d > N$: excess risk decomposes into **variance** + **bias² term**. Second descent occurs as $d \gg N$ (variance $\downarrow$, bias $\uparrow$).
- ▶ As $N \rightarrow \infty$ with fixed d , the risk approaches $\frac{\sigma^2 d}{N}$ (classical rate).



Understanding the Interpolation Peak

At $d \approx N$ (interpolation threshold):

- ▶ The system $\mathbf{X}\theta = \mathbf{y}$ has (almost) a unique solution. At exactly $d = N$, generically a unique solution exists.
- ▶ No flexibility for the algorithm to select among multiple minimizers.
- ▶ Model is forced to fit noise as well as signal.
- ▶ Solution becomes extremely sensitive to data perturbations.

Consequence: The **risk diverges** as $d \rightarrow N$ from either side, producing the **peak** in the double descent curve. The mathematical formulas break down at $d \in \{N - 1, N, N + 1\}$ where required moments don't exist.

❗ For $d > N$, the generalization risk has a U-shaped curve. Can we find its minimizer? See Exercise 11.5.

❗ Could using ridge regression instead remove the peak near $d \approx N$, i.e., does regularization suppresses the variance explosion? See Exercise 11.4 (b)-(c).

When Does Double Descent Occur?

💡 Double descent emerges from the **interaction between the properties of the data** (distribution, covariance structure, noise level) and the **inductive biases of the learning algorithm** (e.g., GD $\rightarrow$ minimum-norm interpolator).

Important Caveats:

- ▶ Double descent is not universal — it depends on both data and algorithm.
- ▶ It arises specifically for the **minimum-norm interpolator** found by (stochastic) GD.
- ▶ If a different empirical risk minimizer is chosen, the phenomenon may not appear.
- ▶ If “model size” is measured differently (e.g., parameter norm instead of raw dimension), the curve may look different.
- ▶ In fact, one can even **design the generalization curve** (see Exercise 12.6)!

💡 Double descent is a phenomenon, not a law — its emergence depends on the interplay between data, architecture, and optimization.

Multiple Types and Contexts of Double Descent

Nakkiran et. al. (2019): Double descent can appear along multiple axes³:

1. **Model-wise:** Varying number of parameters
2. **Epoch-wise:** Varying training time
3. **Sample-wise:** Varying training set size

Empirical evidence across contexts:

- ▶ Linear regression with Gaussian design and noise
- ▶ Random feature models and kernel regression
- ▶ Deep neural networks (CNNs, transformers)
- ▶ Ensemble methods (e.g., random forests)

💡 Double descent is *not at odds* with the classical tradeoff: the spike near interpolation reflects variance blow-up at the boundary of model capacity. Beyond this regime, overparameterization and implicit bias allow the variance to shrink again.

³See <https://openai.com/index/deep-double-descent/> for a quick intro.

Demonstration: Double Descent with Ridge Regularization

With stronger/optimally tuned L_2 (ridge) regularization, the sharp interpolation peak is *diminished or disappears*, showing how explicit control can smooth out double descent.

Setting (see the code): Linear model with Gaussian features

$$y = X\beta^* + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I),$$

where $\|\beta^*\|_2 = 1$ is fixed.

Double Descent with L2 Regularization ($\sigma=0.5$, $\|\beta\|_2=1.0$)



From Double Descent to Benign Overfitting

Key puzzle: Why does test error decrease again when $d \gg N$, even though the model interpolates the noise?

- ▶ In high dimensions, the minimum-norm interpolator $\hat{\theta}$ aligns mostly with the signal subspace.
- ▶ Noise is fit in directions *orthogonal* (or nearly orthogonal) to θ^* .
- ▶ These directions have little effect on prediction error.

Benign Overfitting: Overfitting is *benign* when the fitted noise lives in low-signal directions that do not harm generalization.

Practical implication: Overparameterization can sometimes *help* generalization, especially when the data has hidden structure.

Benign Overfitting: Why Overparameterization Helps

Two contrasting covariance structures:

- ▶ **Isotropic case** ($\Sigma = I$): As $d \rightarrow \infty$, excess risk $\rightarrow \|\theta^*\|_2^2$. Noise affects all directions equally $\rightarrow$ overfitting is harmful.
- ▶ **Anisotropic case** (decaying spectrum of Σ):
 - ▶ Signal concentrates in top eigendirections (large eigenvalues).
 - ▶ Noise is fit in weak eigendirections (small eigenvalues).
 - ▶ Weak directions contribute negligibly to prediction risk.

💡 Benign overfitting arises from *anisotropy*: noise gets absorbed in low-variance directions, leaving predictions accurate.

Bartlett et. al. (2020) provides a precise analysis along this line in the linear model setting.

Scaling Laws: Empirical Observation

Surprising empirical fact: In many domains, larger models consistently perform better, *even when already overparameterized*.

- ▶ **Language models:** GPT-1 (117M) → GPT-2 (1.5B) → GPT-3 (175B) → GPT-4 (~1.7T).
- ▶ **Computer Vision:** Increasing depth and width (in, e.g., Transformers) improves accuracy.
- ▶ **Scientific ML:** Larger physics-informed NNs often yield better accuracy.

Kaplan et. al. (2020): Study empirical scaling laws for language model performance on the cross-entropy loss, and show that performance follows approximate **power-law scaling**:

$$\text{Error or Loss} \propto (\text{Model Size})^{-\alpha} \times (\text{Data Size})^{-\beta} \times (\text{Compute})^{-\gamma}.$$

Scaling Laws for Neural Language Models

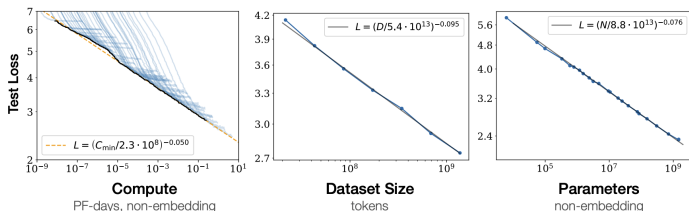


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

- ▶ Exponents $\alpha, \beta, \gamma > 0$ depend on task and architecture.
- ▶ Not universal across all ML domains, but holds remarkably well for language modeling.
- ▶ Suggests that we have not yet hit diminishing returns. Performance often improves predictably with more parameters, more data, and more compute.
- ▶ Open problems: when do the laws break down? how do they depend on data quality vs. quantity? how can we understand the empirical laws?

Active Research Directions

Many open research directions, including but not limited to understanding:

- ▶ **Mechanistic Interpretability**⁴: Understanding what neural networks actually learn and why
- ▶ **Lottery Ticket Hypothesis**⁵: Finding sparse subnetworks that train as well as full models
- ▶ **Neural Scaling Laws**: Predicting performance from compute, data, and parameters.
- ▶ **Grokking**⁶: Understanding sudden generalization after memorization

The field is rapidly evolving. New discoveries regularly challenge our understanding of deep learning. *Theory is lagging behind experiments.*

⁴See <https://www.anthropic.com/research/tracing-thoughts-language-model> for a fun read.

⁵See <https://arxiv.org/abs/1803.03635>.

⁶See <https://arxiv.org/abs/2201.02177>.

Key Takeaways

1. **Regularization Everywhere:** Regularization (explicit *or* implicit) is present in all regimes. In the overparameterized setting, it is *crucial* for explaining why interpolation can still generalize.
2. **Classical Theory Refined:** The bias–variance tradeoff alone is incomplete. Modern results extend it: test error can spike at the interpolation threshold and then decrease again (*double descent*). The curve depends on how model complexity is measured (dimension vs. norm). In controlled settings, one can even engineer *multiple descents*.
3. **Overparameterization Can Help:** Having more parameters can improve generalization, thanks to implicit biases of optimization.
4. **Optimization Matters:** Gradient descent does not pick an arbitrary interpolator — it converges to the minimum-norm solution. Algorithm choice shapes generalization.
5. **Practice Leads Theory:** Phenomena like benign overfitting and scaling laws were observed empirically before being partially explained theoretically⁷.

⁷<https://arxiv.org/abs/2105.04026>.

Key Papers and References

- ▶ Zhang et al. (2017): [Understanding deep learning requires rethinking generalization](#) → DNNs can fit random labels yet generalize on real data.
- ▶ Belkin et al. (2019): [Reconciling modern ML practice and the bias–variance trade-off](#) → Popularize the double descent phenomenon.
- ▶ Nakkiran et al. (2019): [Deep Double Descent](#) → Empirical evidence in deep networks.
- ▶ Bartlett, Long, Lugosi, & Tsigler (2020): [Benign overfitting in linear regression](#) → Why interpolation can still generalize.
- ▶ Hastie, Montanari, Rosset, & Tibshirani (2022): [Surprises in High-Dimensional Ridgeless Least Squares](#).
- ▶ Mei & Montanari (2022): [Generalization error of random features and NTK regression](#).
- ▶ D'Ascoli et al. (2020): [Triple descent and the two kinds of overfitting](#).
- ▶ Nakkiran et al. (2020): [Optimal Regularization Can Be Data-Dependent](#) → Designing generalization curves.

Exercise: Implicit Regularization in Gradient Descent

Exercise 11

Consider linear regression with squared loss

$$L(\theta) = \frac{1}{2} \|\mathbf{X}\theta - \mathbf{y}\|_2^2, \quad \mathbf{X} \in \mathbb{R}^{N \times d}, \mathbf{y} \in \mathbb{R}^N.$$

- (a) Write down the gradient descent update for θ and state the stability condition on the step size η .
- (b) Analyze the dynamics of gradient descent by expressing the iterates in the eigenbasis of $A = \mathbf{X}^\top \mathbf{X}$. What is the effect of η and the iteration number t on each eigendirection?
- (c) Explain why early stopping acts as a form of *spectral regularization*, and compare this behavior to ridge regression.

Exercise: Noise Injection as Explicit Regularization

Exercise 11

2. Consider training with Gaussian input noise:

$$\tilde{x} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I_d).$$

For a loss function ℓ and predictor $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}$, the noise-regularized risk is

$$R_{\text{noise}}(\theta) = \mathbb{E}_{(x,y)} \mathbb{E}_\varepsilon \ell(f_\theta(x + \varepsilon), y).$$

- (a) Assume that the function $g(x) := \ell(f_\theta(x), y)$ is twice continuously differentiable in x . Using a second-order Taylor expansion in ε , show that

$$R_{\text{noise}}(\theta) \approx R(\theta) + \frac{\sigma^2}{2} \mathbb{E}_{(x,y)} [\text{tr}(H_x \ell(f_\theta(x), y))],$$

where $R(\theta) = \mathbb{E}_{(x,y)} [\ell(f_\theta(x), y)]$ is the original risk and $H_x g(x)$ denotes the Hessian of $g(x)$ with respect to x .

- (b) Specialize to the squared-error loss $\ell(f, y) = \frac{1}{2}(f - y)^2$ and derive the correction term.

Exercise: Mixup as Implicit Regularization

Exercise 11

3. Consider the Vicinal Risk Minimization (VRM) objective, which trains on convex interpolations between points x, x' drawn from the same class^a:

$$\mathcal{L}_{\text{VRM}}(\theta) = \mathbb{E}_{(x,y)} \mathbb{E}_{\lambda \sim \text{Uniform}[0,1]} \mathbb{E}_{x' \sim p(\cdot|y)} \ell(f_{\theta}(x + \lambda(x' - x)), y).$$

- (a) Assume $g(x) := \ell(f_{\theta}(x), y)$ is twice continuously differentiable and specialize to the squared loss. By applying a second-order Taylor expansion with respect to the perturbation $\delta = \lambda(x' - x)$, show that

$$\mathcal{L}_{\text{VRM}}(\theta) \approx \mathcal{L}_{\text{ERM}}(\theta) + \frac{1}{3} \mathbb{E}_{(x,y)} [\nabla_x f_{\theta}(x)^{\top} \Sigma_y \nabla_x f_{\theta}(x)],$$

where $\mathcal{L}_{\text{ERM}}(\theta) = \mathbb{E}_{(x,y)} [\ell(f_{\theta}(x), y)]$ and $\Sigma_y = \mathbb{E}_{x'|y} [(x' - x)(x' - x)^{\top}]$ is the class-conditional covariance matrix.

- (b) Compare this penalty with the noise injection regularizer from the previous exercise. What is the key difference in the directions along which smoothness is enforced?

^aIn standard mixup (Zhang et al., 2018); <https://arxiv.org/abs/1710.09412>, pairs $(x, y), (x', y')$ are drawn *regardless of class*, and both inputs and labels are interpolated: $\tilde{x} = \lambda x + (1 - \lambda)x', \tilde{y} = \lambda y + (1 - \lambda)y'$. Here, we restrict to $x' \sim p(\cdot|y)$ (same class) so that labels remain unchanged for simplicity.

Exercise: Risk of OLS and Ridge under Random Design

Exercise 11

4. Consider the random-design linear model

$$y = x^\top \theta^* + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2), \quad x \sim P \text{ with covariance } \Sigma = \mathbb{E}[xx^\top],$$

where Σ is positive definite. We observe N i.i.d. samples $\{(x_i, y_i)\}_{i=1}^N$, and define the empirical covariance $\hat{\Sigma} = \frac{1}{N} \sum_{i=1}^N x_i x_i^\top$. Let $\hat{\theta}_{\text{OLS}}$ denote the OLS estimator and $\hat{\theta}_\lambda$ the ridge estimator with $\lambda > 0$. The Bayes risk is $R^* = \sigma^2$.

- (a) Prove that for OLS, we have: $\mathbb{E}[R(\hat{\theta}_{\text{OLS}})] - R^* = \frac{\sigma^2}{N} \mathbb{E}[\text{tr}(\Sigma \hat{\Sigma}^{-1})]$.
- (b) Show that for ridge regression, we have: $\mathbb{E}[R(\hat{\theta}_\lambda)] - R^* = \lambda^2 \mathbb{E}[\theta^{*\top} (\hat{\Sigma} + \lambda I)^{-1} \Sigma (\hat{\Sigma} + \lambda I)^{-1} \theta^*] + \frac{\sigma^2}{N} \mathbb{E}[\text{tr}((\hat{\Sigma} + \lambda I)^{-2} \hat{\Sigma} \Sigma)]$.
- (c) Discuss whether a double-descent peak occurs as d crosses N for ridge with fixed $\lambda > 0$. Use the expression in (b) and consider the behavior of the eigenvalues of $\hat{\Sigma}$ when $d > N$. What changes when $\lambda \rightarrow 0$?

Exercise: U-shaped Excess Risk for $d > N$

Exercise 11

5. From the Double Descent Theorem (Theorem 2), we have, for $d > N + 1$,

$$E_r(d) := \mathbb{E}[R(\hat{\theta})] - R^* = \underbrace{\left(1 - \frac{N}{d}\right) \|\theta^*\|^2}_{\text{bias term}} + \underbrace{\sigma^2 \cdot \frac{N}{d-N-1}}_{\text{variance term}}.$$

Define the signal-to-noise ratio (SNR): $\text{SNR} = \frac{\|\theta^*\|^2}{\sigma^2}$.

- (a) Determine when $E_r(d)$ admits a critical point in $(N + 1, \infty)$. What condition on SNR is required?
- (b) Assuming such a critical point exists, find the minimizer d^* in terms of N and SNR.
- (c) Discuss limiting behaviors:
 - (i) As $d \downarrow N + 1$, which term dominates?
 - (ii) What is $\lim_{d \rightarrow \infty} E_r(d)$?
 - (iii) For which SNR does a finite minimizer d^* exist? What happens to d^* as $\text{SNR} \rightarrow 1^+$ and as $\text{SNR} \rightarrow \infty$?

Exercise: Design Your Own Generalization Curve!

Exercise 11

6. **[Experimental]** In classical linear regression, the *double descent* curve appears when test error decreases for $d < N$, spikes near $d = N$, and decreases again for $d > N$. Here we study the *feature-revealing* construction^a, where the distribution of newly added features can be chosen. This allows us to engineer *multiple descents*.

Generate training data with $N = 50$ samples and $D = 100$ features, with $\theta^* = 0$, $\varepsilon_i \sim \mathcal{N}(0, 1)$, as follows:

- ▶ For the first 58 features ($d = 1, \dots, 58$): $x_{i,d} \sim \mathcal{N}(0, 1)$.
- ▶ For the next 6 features ($d = 59, \dots, 64$): alternate between *benign* and *toxic* according to

$$\Delta = \{\downarrow, \uparrow, \downarrow, \downarrow, \uparrow, \downarrow\}.$$

where

- ▶ **Benign feature:** $x_{i,d} \sim \mathcal{N}(0, \sigma^2)$.
- ▶ **Toxic feature:** $x_{i,d} \sim \frac{1}{3}\mathcal{N}(-\mu, \sigma^2) + \frac{1}{3}\mathcal{N}(0, \sigma^2) + \frac{1}{3}\mathcal{N}(+\mu, \sigma^2)$.
- ▶ For the remaining features ($d = 65, \dots, 100$): $x_{i,d} \sim \mathcal{N}(0, 1)$.

^aSee <https://arxiv.org/abs/2008.01036>.

Exercise: Design Your Own Generalization Curve!

Exercise 11

Try the following for different values of μ and σ .

- (a) For each d , fit the minimum-norm interpolator

$$\hat{\theta}_d = \mathbf{X}_d^+ \mathbf{y}$$

where $\mathbf{X}_d$ is the design matrix restricted to the first d features.

- (b) To evaluate the models, first generate a large, independent test set (e.g., $n_{\text{test}} = 2000$ samples). For each test sample, generate the full $D = 100$ dimensional feature vector and the corresponding label y_i using the process described above. Then, for each estimator $\hat{\theta}_d$, calculate the excess test risk:

$$L_d = \frac{1}{n_{\text{test}}} \sum_i (\mathbf{x}_i^\top \hat{\theta}_d - y_i)^2 - 1.$$

- (c) Plot L_d for $d \in [45, 70]$ and observe whether the test error exhibits descents and ascents that match the feature pattern $\Delta = \{\downarrow, \uparrow, \downarrow, \downarrow, \uparrow, \downarrow\}$. Discuss how this depends on the values of μ and σ used.

Discussion: Double vs. Multiple Descent

- ▶ **Double descent:** With all features i.i.d. $\mathcal{N}(0, 1)$, test error has the universal shape: decrease $\rightarrow$ spike at $d = N \rightarrow$ second descent.
- ▶ **Multiple descent**⁸: By controlling the distribution of revealed features, we can engineer additional peaks and valleys beyond $d = N$.
- ▶ **Out-of-distribution (OOD) connection:** Adding new features from a different distribution is a toy model of distribution shift.
 - ▶ *Benign features* (small-variance Gaussian) $\Rightarrow$ descent.
 - ▶ *Toxic features* (Gaussian mixture with shifted means) $\Rightarrow$ ascent.
- ▶ Generalization curves beyond interpolation are not universal; they depend on how future (possibly OOD) data distributions differ from the training setup!

💡 This connects to our next lecture on *probabilistic approaches*, where we study how modeling uncertainty can help reason about generalization and OOD case.

⁸<https://arxiv.org/abs/2008.01036>.

Appendix: Auxiliary Results

1D case: If $z_1, \dots, z_N \sim \mathcal{N}(0, 1)$ i.i.d., then

$$Q = \sum_{i=1}^N z_i^2 \sim \chi_N^2.$$

So the χ^2 distribution describes the sampling variability of variance in 1D.

Multi-D case: If $\mathbf{x}_i \sim \mathcal{N}(0, I_d)$ i.i.d., then

$$S = \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^\top = \mathbf{X}^\top \mathbf{X}$$

is a random $d \times d$ positive semidefinite matrix.

Wishart distribution: The multivariate analogue of χ^2 , governing the distribution of covariance matrices.

Definition: Wishart Distribution⁹

Definition 1: Wishart Distribution

Let $x_1, \dots, x_N \sim \mathcal{N}(0, \Sigma)$ i.i.d. in $\mathbb{R}^d$, and define $S = \sum_{i=1}^N x_i x_i^\top \in \mathbb{R}^{d \times d}$. Then S is said to follow a **Wishart distribution** with parameters (Σ, N) :

$$S \sim \mathcal{W}_d(\Sigma, N),$$

with the density:

$$p(S) = \frac{|S|^{\frac{N-d-1}{2}} \exp\left(-\frac{1}{2}\text{tr}(\Sigma^{-1}S)\right)}{2^{Nd/2} |\Sigma|^{N/2} \Gamma_d(N/2)}, \quad S \succeq 0,$$

where $\Gamma_d(a) = \pi^{\frac{d(d-1)}{4}} \prod_{j=1}^d \Gamma\left(a - \frac{j-1}{2}\right)$ is the multivariate Gamma function, and $|S|$ is the determinant of S .

⁹See, e.g., https://swnydick.github.io/assets/reports/Wishart_Distribution.pdf for details.

Definition: Inverse-Wishart Distribution

Definition 2: Inverse-Wishart Distribution

A random matrix $\Sigma \in \mathbb{R}^{d \times d}$ follows an **inverse-Wishart distribution** with degrees of freedom ν and scale matrix $\Psi \succ 0$, denoted

$$\Sigma \sim \text{IW}_d(\nu, \Psi),$$

if $\Sigma^{-1} \sim \mathcal{W}_d(\Psi^{-1}, \nu)$. The density is:

$$p(\Sigma) = \frac{|\Psi|^{\nu/2} |\Sigma|^{-(\nu+d+1)/2} \exp\left(-\frac{1}{2} \text{tr}(\Psi \Sigma^{-1})\right)}{2^{\nu d/2} \Gamma_d(\nu/2)}, \quad \Sigma \succ 0.$$

Key properties:

- ▶ Requires $\nu > d - 1$ for a proper distribution.
- ▶ Mean: $\mathbb{E}[\Sigma] = \frac{\Psi}{\nu - d - 1}$ (when $\nu > d + 1$).
- ▶ Mode: $\text{Mode}(\Sigma) = \frac{\Psi}{\nu + d + 1}$.

The Needed Wishart Distribution Facts

Let $\mathbf{X} \in \mathbb{R}^{N \times d}$ with rows $x_i^\top \sim \mathcal{N}(0, I_d)$ i.i.d.

The covariance matrix

$$S = \mathbf{X}^\top \mathbf{X} \sim \mathcal{W}_d(I_d, N)$$

is Wishart distributed. We will take the following result as granted.

Lemma 1: Key Properties

If $S \sim \mathcal{W}_d(I_d, N)$, then we have the following.

- (i) $\mathbb{E}[S] = N I_d$.
- (ii) If $N > d + 1$, then

$$\mathbb{E}[S^{-1}] = \frac{1}{N - d - 1} I_d.$$

This form is used when analyzing the **underparameterized regime** ($d < N$).

*Connection to random matrix theory: In the large- N, d limit with $d/N \rightarrow \gamma$, the eigenvalue distribution of $\frac{1}{N} \mathbf{X}^\top \mathbf{X}$ converges to the **Marchenko–Pastur law**.

The Needed Wishart Distribution Facts

Equivalently, the Gram matrix

$$K = \mathbf{X}\mathbf{X}^\top \sim \mathcal{W}_N(I_N, d)$$

follows a Wishart distribution.

Lemma 2: Key Properties

If $K \sim \mathcal{W}_N(I_N, d)$, then we have the following.

- (i) $\mathbb{E}[K] = d I_N$.
- (ii) If $d > N + 1$, then

$$\mathbb{E}[K^{-1}] = \frac{1}{d - N - 1} I_N.$$

This form is used when analyzing the **overparameterized regime** ($d > N$).

Conjugacy and Statistical Connections

Reminder from tutorial: We studied conjugacy in Bayesian estimation (Normal–Normal for the mean). Here, a parallel arises in higher dimensions.

Aspect	1D Mean Estimation	Multi-D Covariance Estimation
Parameter	Scalar θ	Covariance Σ
Likelihood	$y \sim \mathcal{N}(\theta, \sigma^2)$	$\mathbf{x}_i \sim \mathcal{N}(\mathbf{0}, \Sigma)$
Conjugate Prior	Normal	Inverse-Wishart
Data Statistic	Sample mean	Sample covariance matrix

- ▶ In both cases, the conjugate prior ensures closed-form posteriors.
- ▶ For covariance matrices, the **inverse-Wishart prior** is conjugate to the Gaussian likelihood (assuming known or integrated-out mean).
- ▶ The inverse-Wishart prior $\text{IW}(\nu_0, \Psi)$ requires degrees of freedom $\nu_0 > d - 1$ (where d is the dimension) for a proper distribution.
- ▶ As $N \rightarrow \infty$, the posterior concentrates at the empirical statistic (**data dominates the prior**).

💡 Wishart/inverse-Wishart distributions used in our double descent analysis also appear naturally in Bayesian covariance learning!